

Vidarbha Youth Welfare Society's



Prof. Ram Meghe Institute of Technology & Research Badnera, Amravati (M.S.) 444701

(An Autonomous Institute)

Laboratory Manual

Semester IV

Subject: - Microcontroller And Applications Lab

(Subject code: - 4SX07)

(Academic Year: 2024-2025)

Department of Computer Science & Engg.- Internet of Things

Accredited by NAAC A+

Phone: (0721) -2681246, Fax No. 0721 – 2681337

Website: www.mitra.ac.in

Sant Gadge Baba Amravati University- Amravati.

Vidarbha Youth Welfare Society's
List of Experiments
Department of Industrial Internet Of Things

Subject: -Microcontroller & Applications Lab Semester: -5th

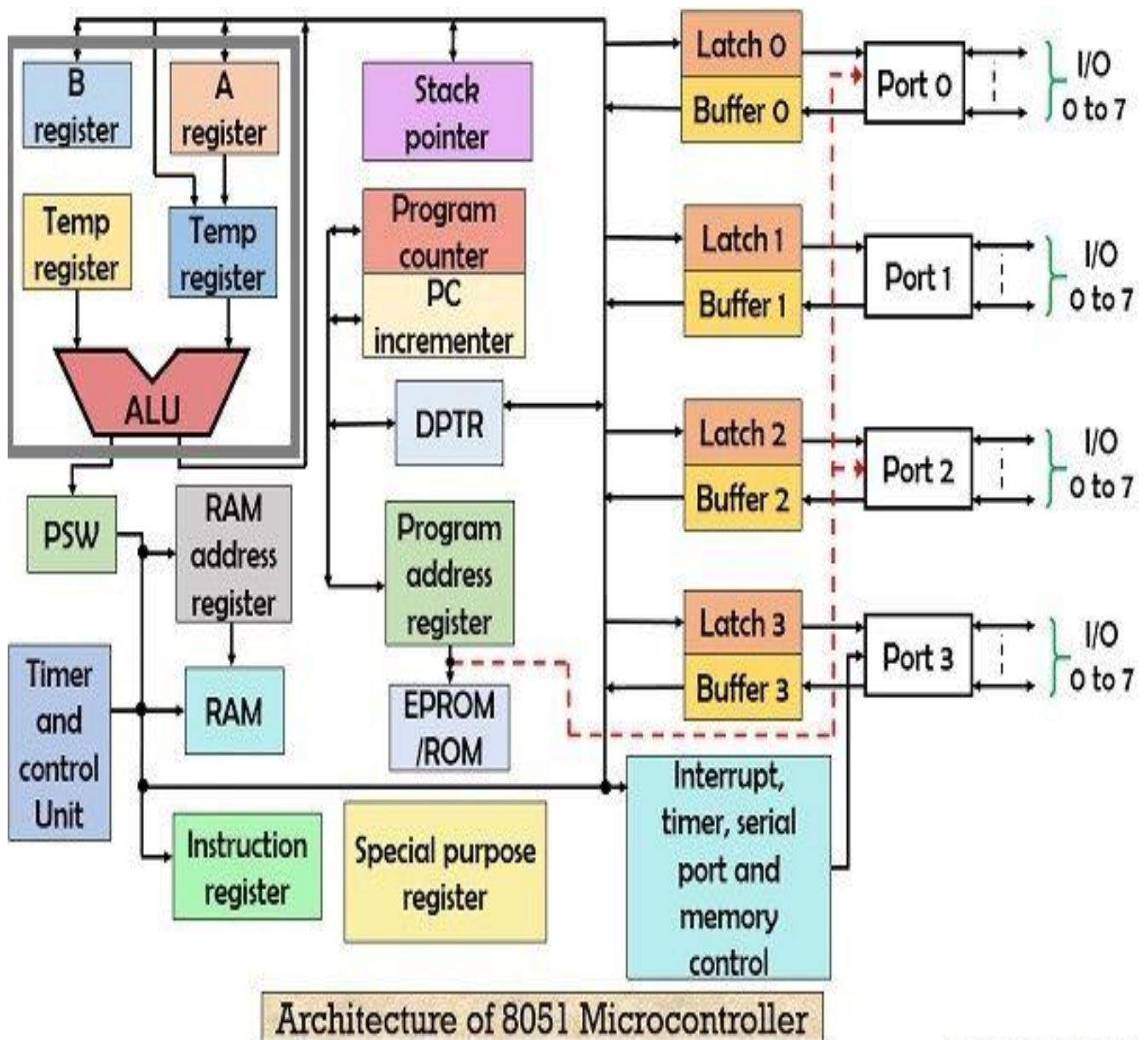
Subject Code: -4SX07

Exp. No.	Aim of experiment
1.	To Study the architecture of 8051 micro-controller.
2.	A. To write and execute an assembly language program for addition of two 8-bit numbers, using 8051 Microcontroller. B. To write and execute an assembly language program for Subtraction of two 8-bit numbers, using 8051 Microcontroller.
3.	A. To write an assembly language program for Multiplication of two 8-bit numbers, using 8051 Microcontroller kit. B. To write an assembly language program for Division of two 8-bit numbers, using 8051 Microcontroller kit.
4.	A. To write and execute an 8051 Assembly Language Program to toggle two alternate hex patterns on Port 1. B. To generate delay using a subroutine.
5.	To interface 7 Segment Display with 8051 Microcontroller.
6.	To interface LEDs and Switches with 8051 Microcontroller.
7.	To transmit a character using serial communication of the 8051 microcontroller.
8.	To interface Stepper motor with 8051 Microcontroller.

EXPERIMENT NO. - 1

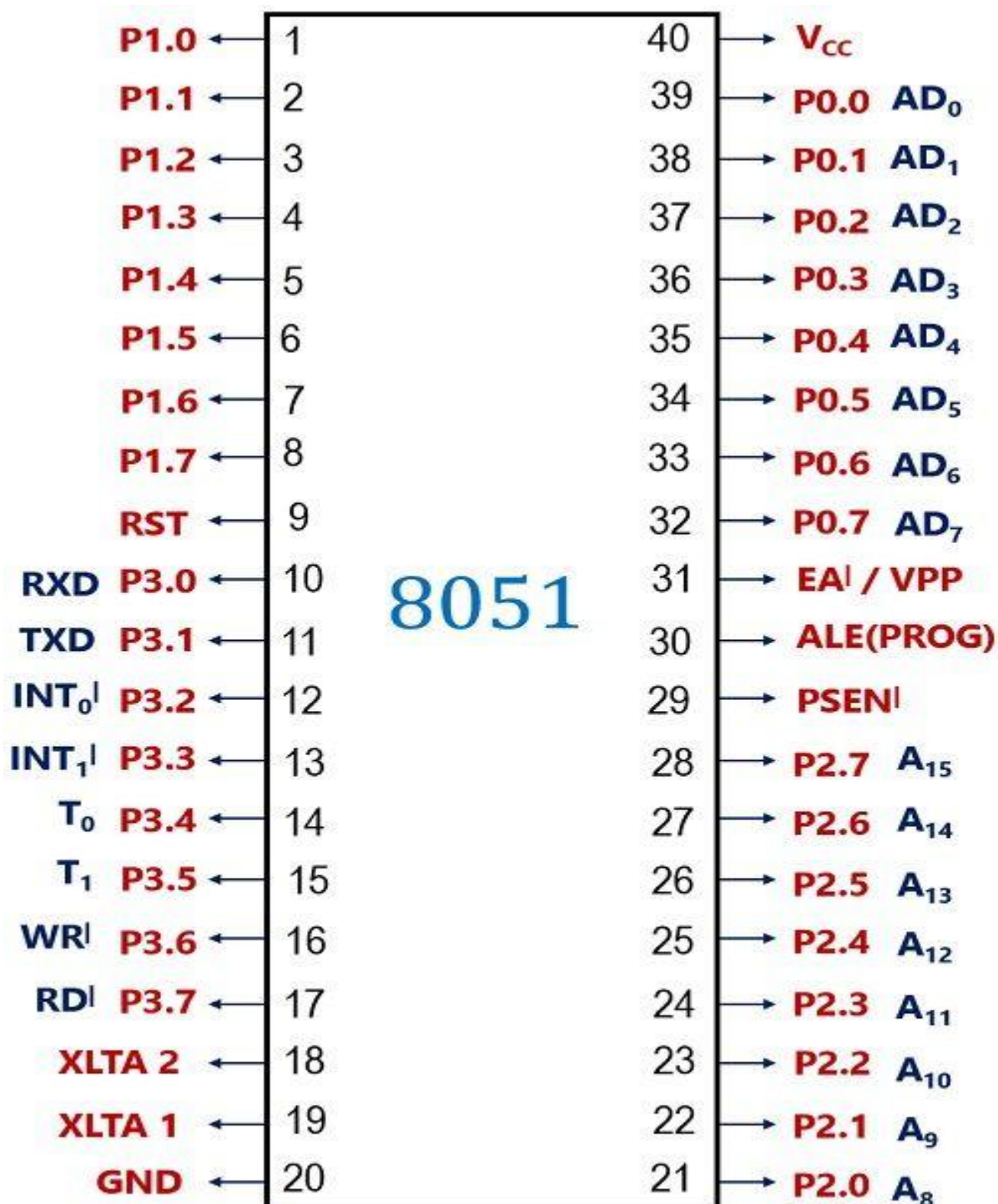
AIM: Study the architecture of 8051 micro-controller.

8051 ARCHITECHTURE:



Electronics Desk

Pin Diagram:



Pin diagram of 8051 Microcontroller

Electronics Desk

Theory –

8051 Architecture:

The 8051 microcontroller is an 8-bit, general-purpose microcontroller developed by Intel. It follows a Harvard architecture, meaning the program memory (ROM/Flash) and data memory (RAM) are separate, allowing faster processing.

Main Architectural Blocks

1. CPU (Central Processing Unit)

- Contains the ALU (Arithmetic & Logic Unit) for calculations and logic operations
- Includes Accumulator (A register), B register, and Program Status Word (PSW) for operation control

2. Memory Organization

- 4 KB on-chip ROM → stores program code
- 128 bytes on-chip RAM → stores temporary data
- Supports external ROM up to 64 KB and external RAM up to 64 KB

3. Registers

- General Purpose Registers: 32 registers arranged in 4 banks (R0–R7 each)
- Special Function Registers (SFRs): control ports, timers, interrupts, serial communication, etc.
- Examples of SFRs:
- A (Accumulator), B register, PSW, P0–P3 (Ports), TMOD, TCON, SBUF, IE, IP

4. I/O Ports (Port 0, 1, 2, 3)

- Total 32 I/O pins
- Each port is 8-bit and bit-addressable

- Port 0 can also work as address/data bus for external memory
- Port 3 has special functions like:
- P3.0–RXD, P3.1–TXD (Serial)
- P3.2–INT0, P3.3–INT1 (Interrupts)
- P3.4–T0, P3.5–T1 (Timers)

5. Timers/Counters

- Two 16-bit timers: Timer 0 and Timer 1
- Controlled by TMOD (mode register) and TCON (control register)

6. Interrupt System

- 8051 supports 5 interrupt sources
- Controlled by IE (Interrupt Enable) and IP (Interrupt Priority) registers
- Interrupts improve real-time response

7. Serial Communication (UART)

- Uses registers SBUF (Serial Buffer) and SCON (Serial Control)
- Communicates through RXD/TXD pins

8. Oscillator & Clock

- External crystal oscillator (typically 11.0592 MHz or 12 MHz) provides the clock
- Controls the speed of execution

Important Features

- 8-bit microcontroller
- Separate code & data memory (Harvard)

- 4 register banks
- 2 timers, 5 interrupts
- 1 UART serial port
- Bit-addressable ports & RAM

Conclusion:

Studying 8051 architecture helps us understand:

- how the CPU processes data
- how memory and registers are organized
- how ports and peripherals (timers, interrupts, serial) interact with external devices|

Questions:

1. What is Harvard architecture in 8051?
2. Explain the role of ALU, Accumulator and PSW in 8051 CPU.
3. Describe internal memory organization of 8051.

Experiment No . 2

Addition and Subtraction of two 8-bit numbers.

Aim: 1. To write and execute an assembly language program for addition of two 8-bit numbers, using 8051 Microcontroller.

2. To write and execute an assembly language program for Subtraction of two 8-bit numbers, using 8051 Microcontroller.

Software Used: Keil Micro vision 5

Theory:

The 8051 is an 8-bit microcontroller that performs arithmetic operations using the ALU (Arithmetic and Logic Unit).

For addition, 8-bit numbers are commonly added using the Accumulator (A register) along with another operand stored in a general purpose register (e.g., Rn) or memory. The result is stored in the Accumulator. If the addition generates a carry beyond 8 bits, the Carry flag (CY) in the PSW register is set.

For subtraction, 8051 performs the operation using 2's complement arithmetic internally. The number in the Accumulator is subtracted with another 8-bit operand. The result remains in the Accumulator. If a borrow is required (when the subtracted number is larger), the Carry flag is set (CY = 1 indicates borrow in subtraction).

Algorithm:

Algorithm for Addition of Two 8-bit Numbers

1. Start
2. Load first 8-bit number into Accumulator (A)
3. Load second 8-bit number into a register (Rn)
4. Add register to accumulator using: ADD A, Rn
5. Result is stored in A
6. Check CY flag in PSW for carry (if generated)
7. En

Program:

ORG 0000H

MOV R0, #25H ; First number

MOV R1, #34H ; Second number

MOV A, R0

ADD A, R1

MOV R2, A ; Store result in R2

HERE: SJMP HERE

END

Algorithm for Subtraction of Two 8-bit Numbers

1. Start
2. Load first 8-bit number into Accumulator (A)
3. Load second 8-bit number into a register (Rn)
4. Clear carry before subtraction: CLR C
5. Subtract register from accumulator using: SUBB A, Rn
6. Result is stored in A
7. Check CY flag for borrow (CY=1 means borrow occurred)
8. End

Program:

ORG 0000H

MOV R0, #50H ; First number

MOV R1, #23H ; Second number

MOV A, R0

CLR C

SUBB A, R1

MOV R2, A ; Store result in R2

HERE: SJMP HERE

END

Result:

Conclusion:

Questions:

1. Which register is primarily used to store the result of 8-bit addition and subtraction in 8051?
2. Why is the CLR C instruction used before performing subtraction using SUBB?
3. What does the Carry flag (CY) represent after an 8-bit addition operation?
4. What does CY indicate during a subtraction operation if it becomes 1 after SUBB?

Experiment No.: 03

Multiplication and Division of two 8-bit numbers.

Aim: 1. To write an assembly language program for Multiplication of two 8-bit numbers, using 8051 Microcontroller kit.

2. To write an assembly language program for Division of two 8-bit numbers, using 8051 Microcontroller kit.

Software Used: Keil Micro vision 5

Theory

The 8051 performs multiplication and division using the ALU with help of special registers:

Multiplication

- Uses instruction: MUL AB
- 8051 multiplies the value in A and B
- Result is 16-bit
 - Lower 8 bits → A
 - Higher 8 bits → B
- CY flag is always cleared after MUL

Division

- Uses instruction: DIV AB
- 8051 divides:
- $A \div B$
- After division:
 - Quotient → A
 - Remainder → B
- $CY = 1$ if $B = 0$ (division by zero error)

Algorithm for Multiplication

1. Start
2. Load first 8-bit number into A
3. Load second 8-bit number into B
4. Execute MUL AB
5. Result:
 - A = lower byte
 - B = higher byte
6. End

Program:

ORG 0000H

MOV A, #12H ; First number

MOV B, #10H ; Second number

MUL AB

MOV R2, A ; Lower 8-bit result

MOV R3, B ; Higher 8-bit result

HERE: SJMP HERE

END

Algorithm for Division

1. Start
2. Load first 8-bit number into A
3. Load second 8-bit number into B
4. Check if B = 0:

- If yes → CY flag set, stop (error case)

5. Execute DIV AB

6. Result:

- A = quotient
- B = remainder

7. End

Program:

ORG 0000H

MOV R0, #64H ; Dividend

MOV R1, #08H ; Divisor

MOV A, R0

MOV B, R1

CJNE R1, #00H, DIVIDE ; Check divisor $\neq$ 0

SETB C ; If 0, set CY flag (error)

SJMP HOLD

DIVIDE: DIV AB ; A=Quotient, B=Remainder

HOLD: MOV R2, A ; Store quotient

MOV R3, B ; Store remainder

HERE: SJMP HERE

END

Result:

Conclusion:

Questions:

1. Which instruction is used for multiplying two 8-bit numbers in 8051, and where is the result stored?
2. After executing DIV AB, which register holds the quotient and which holds the remainder?
3. What happens to the CY flag if division by zero is attempted using DIV AB?
4. Why are A and B registers specifically used for multiplication and division operations?

Experiment No. 4

Toggling Alternate Hex Patterns on Port 1 using Delay Subroutine in 8051 Assembly Language

Aim:

- A. To write and execute an 8051 Assembly Language Program to **toggle** two alternate hex patterns on Port 1.
- B. To generate delay using a subroutine in Keil Micro Vision 5.

Software Used: Keil Micro Vision 5

Theory:

The 8051 microcontroller performs arithmetic and logical operations through the **ALU** and communicates with external devices using 4 parallel I/O ports. **Port 1 (P1)** is frequently used for output interfacing.

A delay can be created using nested loops with registers. The **DJNZ** instruction decrements registers to form delay timing. Subroutines in 8051 are called using **ACALL** (within 2K address range), and control returns to the main program using **RET**.

In this experiment, the program **toggles** Port 1 between:

- **AAH (10101010₂)** → Alternate bits HIGH
- **55H (01010101₂)** → Complement alternate bits HIGH

The delay subroutine allows the toggling effect to be observed clearly when interfaced with output devices like LEDs.

Algorithm:

1. Start
2. Jump to START using SJMP
3. Initialize Port 1 with 00H
4. Load AAH into Port 1 → first toggle pattern
5. Call DELAY subroutine

6. Load 55H into Port 1 → second toggle pattern
7. Call DELAY subroutine again
8. Repeat loop using SJMP AGAIN
9. End

Program:

ORG 0000

SJMP START

ORG 30H

START:

MOV P1, #00H

AGAIN:

MOV P1, #0AAH

ACALL DELAY

MOV P1, #55H

ACALL DELAY

SJMP AGAIN

ORG 70H

DELAY:

MOV R2, #08H

BACK2:

MOV R1, #0FFH

BACK1:

MOV R0, #0FFH

BACK:

NOP

DJNZ R0, BACK

DJNZ R1, BACK1

DJNZ R2, BACK2

RET

END

Result:

Conclusion:

Questions:

1. Which port is toggled in this experiment?
2. Which instruction is used to call the delay subroutine?
3. Which instruction is used to return from the subroutine?
4. What is the binary pattern of the first toggled value (AAH)?

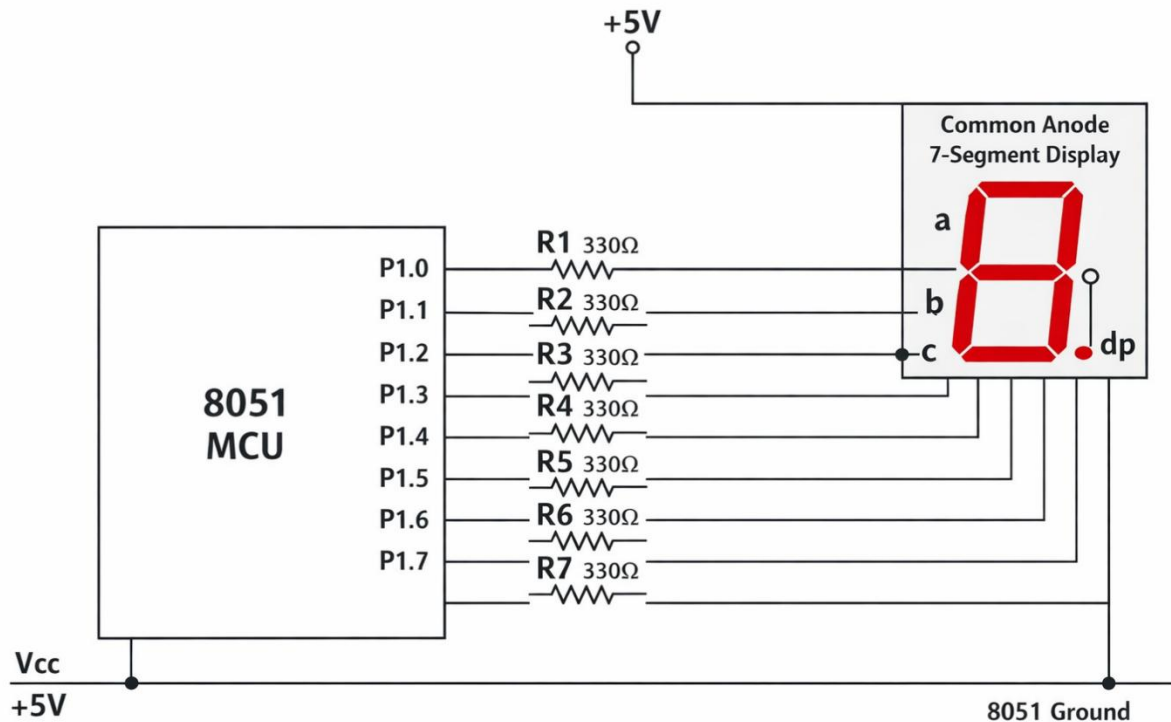
Experiment No . 05

Aim: To interface 7 Segment Display with 8051 Microcontroller.

Apparatus Required: 8051 Microcontroller kit, FRC cables

Software Used: Keil Micro vision 4

Circuit Diagram:



Theory:

A seven-segment display is an electronic display device used to display decimal numerals and some alphabetic characters. It is composed of seven LED (light-emitting diode) segments arranged in a rectangular fashion, as well as an eighth segment for the decimal point. Each segment can be illuminated individually to form various characters.

Components of a Seven-Segment Display

1. Segments (a-g and dp):

- Each segment is an individual LED that can be turned on or off.
- The segments are labeled as a, b, c, d, e, f, and g, arranged in an "8" shape.
- The "dp" segment represents the decimal point.

Types of Seven-Segment Displays

1. Common Cathode:

- All the cathode (negative) terminals of the LEDs are connected together and to ground.
- Each segment is lit by applying a positive voltage to the corresponding segment pin.

2. Common Anode:

- All the anode (positive) terminals of the LEDs are connected together and to the power supply.
- Each segment is lit by applying a ground (0V) to the corresponding segment pin.

Common Anode Configuration:

- In common anode, all LED anodes are internally connected to +5V
- To turn a segment ON, the microcontroller pin must provide LOW (0V) to that segment → this means the 8051 sinks current
- Therefore, the display is driven using active LOW logic

Interfacing with 8051 (Port 1)

In the above circuit:

- **Port 1 pins P1.0 to P1.7** are connected to the 7 segments and dp
- Mapping followed:

P1.0 → a

P1.1 → b

P1.2 → c

P1.3 → d

P1.4 → e

P1.5 → f

P1.6 → g

P1.7 → dp

- Each segment pin is connected through a **current-limiting resistor ($\sim 330\Omega$)** to protect LEDs from excess current
- The display common pin is connected to **+5V supply**
- **8051 GND is shared with display ground reference**

Working Principle

1. 8051 sends data to **Port 1**
2. Since display is common anode, LOW on a pin lights up the corresponding segment
3. Example:

- To display **0**, segments a, b, c, d, e, f must glow → their pins are set LOW, and g pin is set HIGH

4. The controller updates port output to show desired digits

Algorithm:

Algorithm for Displaying Digits/Characters on 7-Segment via 8051

1. Start
2. Configure Port 0 as output (used for digit-select transistor lines in the kit)
3. Configure Port 1 as output (used for 7-segment data lines)
4. Initialize:
 - Set P0 = 0x00 → disable all digit transistors
 - Set P1 = 0xFF → turn OFF all segments (common anode uses active-LOW)
5. Create/declare a function display(byte) to:
 - Send the segment pattern to the port driving the display
 - Call delay to hold the pattern visible
6. Create a delay(time) function using nested empty loops
7. Enter infinite loop
8. Inside loop:
 - Call display() with segment code for 0–9, A–F one by one
9. Repeat loop continuously (pattern changes appear in sequence)
10. End (loop runs forever).

PROCEDURE:

1. Connect +12V supply to board.
2. Connect the USB Cable to the micro controller Kit and select COM port of the PC.
3. Switch ON the power supply.
4. Open ProgISP Flashing Software on PC. Select the IC (AT89S52).
5. Select given .hex File -> click on Load Flash. Select the HEX file. Click on Auto Button. This uploads HEX file into microcontroller shown by green bar.
6. Now if downloading of the program is completed, connect 10 pin FRC cables to the
7. SEVEN SEGMENT section of the DVK kit
8. will observe that Seven segment will Display (0000 To FFFF).

Connections:

PORT 0(J1) To J11(Seven segment)

PORT 1(J2) To J12(Seven Segment)

Result:**Conclusion:****Questions:**

1. Why are current-limiting resistors used while interfacing a 7-segment display with 8051?
2. In a common anode 7-segment display, what logic level must be applied to a segment pin to turn it ON, and why?
3. What is the role of Port 0 transistors in your multiplexed 7-segment display module?

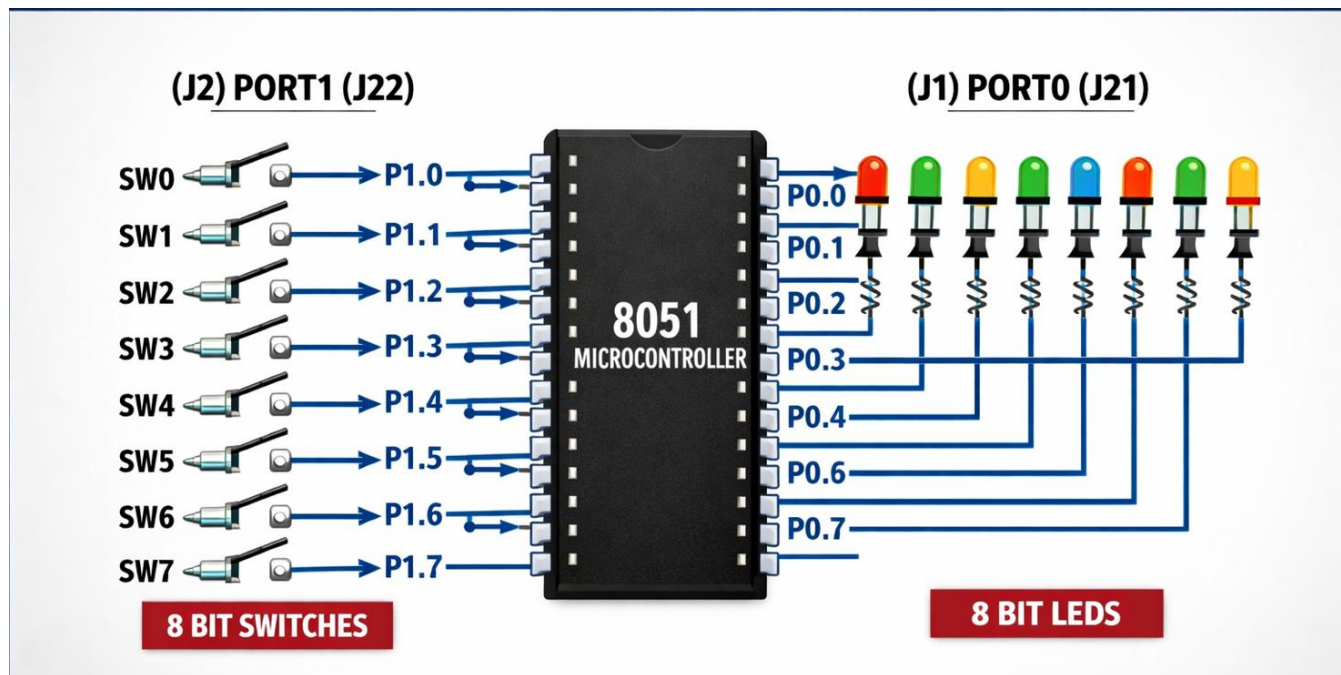
Experiment No . 06

Aim: To interface LED's and Switches with 8051 Microcontroller.

Apparatus Required: 8051 Microcontroller kit, FRC cables

Software Used: Keil Micro vision 5

Circuit Diagram:



Theory:

This circuit shows basic digital I/O interfacing where 8 switches provide input to the 8051 and 8 LEDs display output logic based on the switch states.

- **Port Usage**

PORT1 (P1.0 – P1.7) → connected to 8 switches (input devices)

PORT0 (P0.0 – P0.7) → connected to 8 LEDs (output indicators)

- **Input Configuration (Switches)**

Switches are connected to PORT1 because it has internal pull-ups.

The program sets P1 = 0xFF, making all pins HIGH (1) by default, so the switches work in active-low or high logic input mode depending on wiring.

When a switch is pressed, the microcontroller reads the voltage level at that pin (1 or 0).

Output Configuration (LEDs)

LEDs are connected to PORT0 pins which act as output lines.

The program initializes P0 = 0x00, turning all LEDs OFF initially.

Each LED is controlled individually using sbit (bit-addressable pins) in the code.

- **Working Logic**

The microcontroller continuously checks switch status inside an infinite while loop.

If a switch pin reads 1, the corresponding LED pin is set 1 → LED ON.

If it reads 0, LED is turned OFF.

Example:

- SW0 pressed → P1.0 changes → LED0 at P0.0 glows
- SW7 pressed → LED7 at P0.7 glows

- **Bit Addressability Advantage**

Using sbit allows control of each pin separately without affecting other pins on the port, which is ideal for LED logic display.

- **Real-time Response**

Since the code keeps polling (checking) switch states repeatedly, LEDs update instantly based on user input.

Algorithm:

1. Start
2. Initialize PORT0 as output
3. Turn OFF all LEDs by loading 00H (0x00) into PORT0
4. Initialize PORT1 as input by loading FFH (0xFF) into PORT1
5. Repeat Forever (Loop begins)
6. Read switch SW0 (P1.0) → If it is HIGH (1), turn ON LED0 (P0.0), else turn it OFF

7. Read switch SW1 (P1.1) → If it is HIGH (1), turn ON LED1 (P0.1), else turn it OFF
8. Read switch SW2 (P1.2) → If it is HIGH (1), turn ON LED2 (P0.2), else turn it OFF
9. Read switch SW3 (P1.3) → If it is HIGH (1), turn ON LED3 (P0.3), else turn it OFF
10. Read switch SW4 (P1.4) → If it is HIGH (1), turn ON LED4 (P0.4), else turn it OFF
11. Read switch SW5 (P1.5) → If it is HIGH (1), turn ON LED5 (P0.5), else turn it OFF
12. Read switch SW6 (P1.6) → If it is HIGH (1), turn ON LED6 (P0.6), else turn it OFF
13. Read switch SW7 (P1.7) → If it is HIGH (1), turn ON LED7 (P0.7), else turn it OFF
14. End Loop (go back to step 5)
15. Stop

Procedure:

- 1) Connect +12V supply to board.
- 2) Connect the USB Cable to the micro controller Kit and select COM port of the PC.
- 3) Switch ON the power supply.
- 4) Open ProgISP Flashing Software on PC. Select the IC (AT89S52).
- 5) Select given ledsw hex File -> click on Load Flash. Select the HEX file. Click on Auto Button. This uploads HEX file into microcontroller shown by green bar.
- 6) Connect 10 pin FRC Cables. Press RESET and observe output.

Connections:

PORT 0(J1)

PORT 1(J2)

J21 (Digital I/O)

J22(Digital I/O)

Result:

Conclusion:

Questions:

1. Which port has internal pull-up resistors?
2. Why does PORT0 need external pull-ups when used as I/O?
3. Which port is used for external memory interfacing?

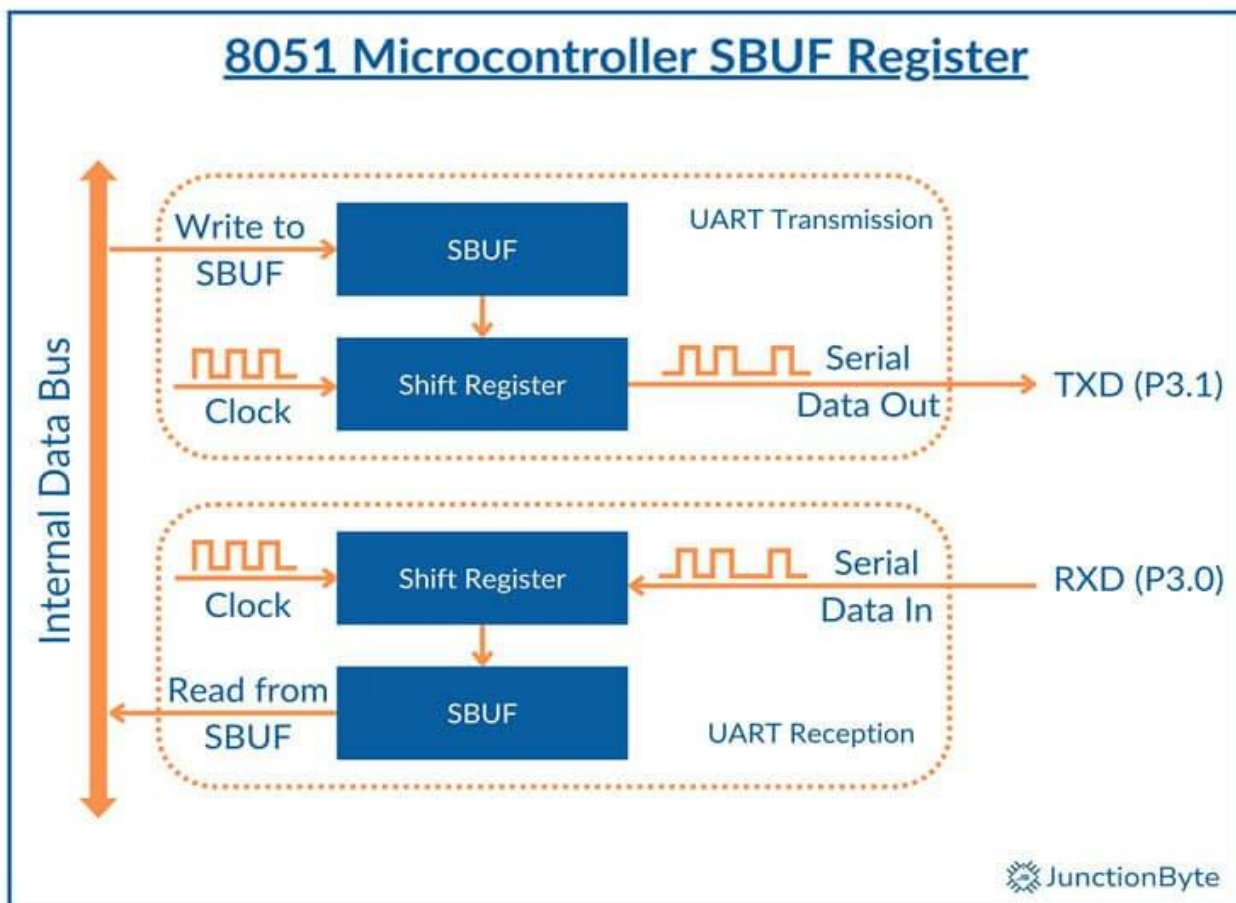
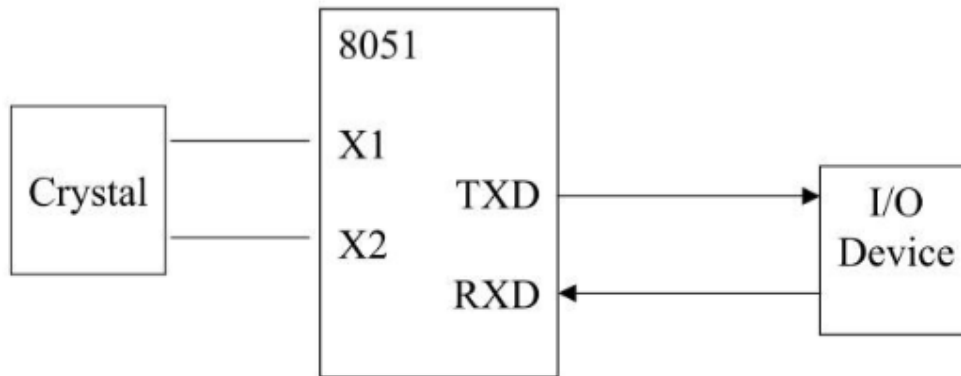
Experiment No. 07

Aim: To transmit a character using serial communication of the 8051 microcontroller.

Apparatus Required: 8051 Microcontroller kit, FRC cables

Software Used: Keil Micro vision 5

Circuit Diagram:



Theory:

Serial communication is a method of transferring data one bit at a time over a single communication line. In microcontroller systems, it is widely used for communication with external devices such as computers, sensors, and other microcontrollers.

The **8051 microcontroller** has a built-in **UART (Universal Asynchronous Receiver/Transmitter)** for serial communication. It supports both **synchronous and asynchronous modes**, but commonly operates in asynchronous mode for most applications.

Registers used for Serial Communication:

1. SBUF (Serial Buffer Register):

- Used to store data for transmission and reception.
- Writing to SBUF sends data; reading from SBUF receives data.

2. SCON (Serial Control Register):

- Controls serial communication modes and status.
- Important bits:
 - SM0, SM1 → Select mode (Mode 0–3)
 - REN → Enable reception
 - TI → Transmit interrupt flag
 - RI → Receive interrupt flag

3. TMOD (Timer Mode Register):

- Used to configure Timer 1 for baud rate generation.

4. TH1 (Timer 1 High Byte):

- Loaded with a value to set the baud rate.

Serial Communication Modes

The 8051 supports four modes:

- **Mode 0:** Shift register mode (fixed baud rate)
- **Mode 1:** 8-bit UART (variable baud rate)
- **Mode 2:** 9-bit UART (fixed baud rate)
- **Mode 3:** 9-bit UART (variable baud rate)

Mode 1 is most commonly used in experiments.

Baud Rate Generation

Baud rate determines the speed of data transmission.

In 8051, baud rate is generated using **Timer 1**.

For Mode 1:

$$\text{Baud Rate} = \frac{\text{OscillatorFrequency}}{32 \times 12 \times (256 - TH1)}$$

Example: For 9600 baud rate, TH1 is loaded accordingly.

Algorithm:

Serial Communication using 8051 (Transmit & Receive)

1. **Start**
2. **Initialize serial communication**
 - Set Timer1 in Mode 2 (auto-reload mode)
 - Load TH1 with value for desired baud rate (e.g., 9600 → FDH)
 - Configure SCON for Mode 1 (8-bit UART) and enable reception
 - Start Timer1
3. **Begin infinite loop**
4. **Wait for data reception**
 - Continuously check RI (Receive Interrupt flag)

- If RI = 1, data is received

5. Read received data

- Store data from SBUF into a variable
- Clear RI flag

6. Transmit data

- Load received data into SBUF
- Wait until TI (Transmit Interrupt flag) becomes 1
- Clear TI flag

7. Repeat steps 4 to 6 continuously

8. Stop

Procedure:

1.Connect +12V supply to board.

2.Connect the USB to the micro controller Kit and select COM port of the PC.

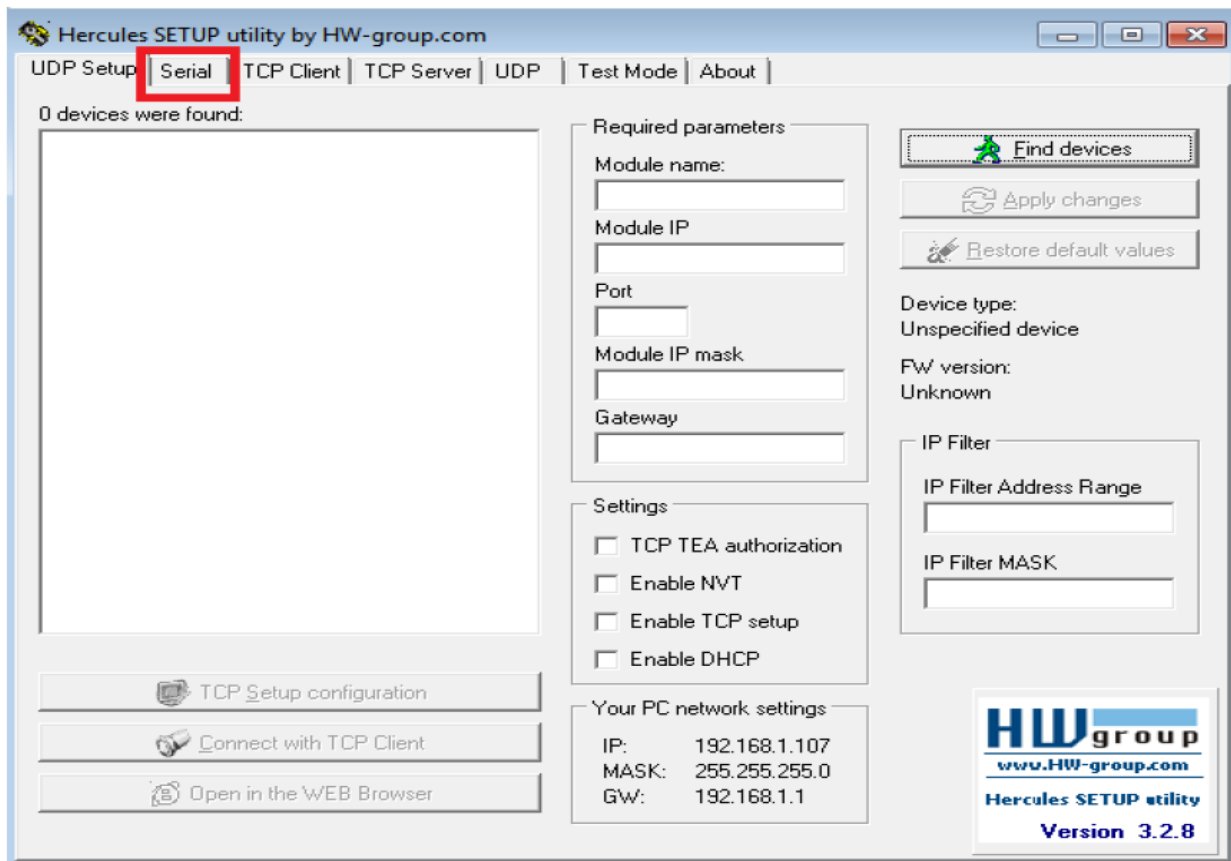
3.Switch ON the power supply.

4.Open ProgISP Flashing Software on PC. Select the IC (AT89S52).

5.Select given .hex File -> click on Load Flash. Select the HEX file. Click on Auto Button. This uploads HEX file into microcontroller shown by green bar.

6.Now if downloading of the program is completed, open Hercules software->

7.Open Hercules CLICK ON SERIAL



Select baud rate 9600, select respective port of your pc/laptop which is connected to dvk

Keep sw3 up and click on open you can see following window as outut.

Connect A-B USB Cable Between DVK and PC/Laptop. upload the program and open Hercules software, go to serial select COM port and set baud rate at 9600,make SW UP and click on open COM. You can see output LOGSUN SYSTEM .

Program:

```
#include <reg51.h>

void serial_init()
{
    TMOD = 0x20;    // Timer1 Mode2
    TH1 = 0xFD;     // 9600 baud rate
    SCON = 0x50;    // Serial mode1, enable receive
    TR1 = 1;        // Start timer1
}

void serial_tx(unsigned char ch)
{
    SBUF = ch;
    while(TI==0);
    TI = 0;
}

unsigned char serial_rx()
{
    while(RI==0);
    RI = 0;
    return SBUF;
}

void main()
{
    unsigned char data;

    serial_init();

    while(1)
    {
        data = serial_rx(); // Receive character
        serial_tx(data);    // Send same character back
    }
}
```


}

}

Result:

Conclusion:

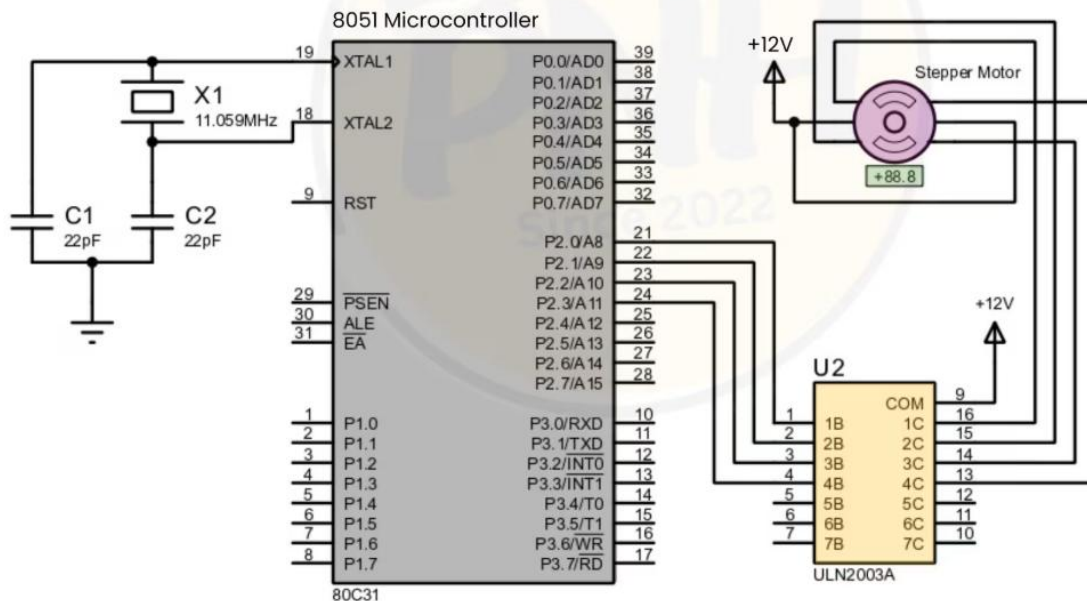
Questions:

Experiment No . 08

Aim: To interface Stepper motor with 8051 Microcontroller.

Appratus: BOARD on AT89S52, 10 pin FRC cable, USB cable, 12V power supply, Stepper motor card.

Circuit Diagram:



Theory:

Basically the stepper motor is an electromagnetic device, which converts digital pulses in to discrete mechanical rotational movements.

In rotary step motor, the output shaft of the motor rotates in equal increments, in response to a train of input pulses.

CONSTRUCTION:

The stepping motor is basically a motor with two phases, eight silent poles, toothed iron rotor and a permanent magnet. This rotor is known as hybrid rotor.

The rotor is suspended in the stator by means of sealed ball bearings.

WORKING OF STEPPING MOTOR:

Sequential switching of supply to the two phases of motor causes the stepping action.

The stepping motor can be run in two modes,

a) Full Step Mode: In this mode out of the four coils, two coils are energized at each time when we apply the pulses. The step angle in this case is $7.5^\circ \pm 0.1^\circ$. The speed of the motor can vary by varying the time duration between the two pulses.

Applying the pulses sequence in the reverse can change the direction of the motor

Direction, that is from BOTTOM to TOP.

b) Half step Mode: In this mode, we get the step angle of 3.75 degree, hence better resolution.

Algorithm:

Algorithm: Stepper Motor Interfacing with 8051 (CW & CCW Rotation)

1. **Start**
2. **Initialize Port**
 - Configure Port 1 as output (default in 8051)
3. **Begin infinite loop**
4. **Rotate motor in Clockwise direction**
 - Send sequence 09H to Port 1
 - Call delay
 - Send sequence 0CH to Port 1
 - Call delay
 - Send sequence 06H to Port 1
 - Call delay
 - Send sequence 03H to Port 1
 - Call delay
5. **Rotate motor in Anticlockwise direction**
 - Send sequence 03H to Port 1
 - Call delay
 - Send sequence 06H to Port 1
 - Call delay
 - Send sequence 0CH to Port 1

- Call delay
- Send sequence 09H to Port 1
- Call delay

6. Repeat steps 4 and 5 continuously

7. Stop

PROCEDURE FOR STEPPER MOTOR:

- 1.Connect +12V supply to board.
- 2.Connect the USB to the micro controller Kit and select COM port of the PC.
- 3.Switch ON the power supply.
- 4.Open ProgISP Flashing Software on PC. Select the IC (AT89S52).
- 5.Select given .hex File -> click on Load Flash. Select the HEX file. Click on Auto Button. This uploads HEX file into microcontroller shown by green bar.
- 6.Connect 10 pin FRC Cables to the stepper motor section of the DVK kit.
- 7.Press start switch then increment. You can change the direction of motor by pressing REV switch for keyboard. You can start or stop the motor by pressing the START or STOP keys.

Connect Motor card at RL4 on DVK BOARD.

Jumpers Settings For stepper motor :

JP6, JP7 – 2 & 3 short.

Program:

```
#include <reg51.h>
```

```
void delay()
```

```
{  
    int i,j;  
    for(i=0;i<300;i++)  
        for(j=0;j<300;j++);  
}
```

```
void main()
```

```
{  
    while(1)  
    {  
        // Clockwise Rotation  
        P1 = 0x09;  
        delay();  
        P1 = 0x0C;  
        delay();  
        P1 = 0x06;  
        delay();  
        P1 = 0x03;  
        delay();  
  
        // Anticlockwise Rotation  
        P1 = 0x03;  
        delay();  
        P1 = 0x06;  
        delay();  
        P1 = 0x0C;  
        delay();  
        P1 = 0x09;  
        delay();  
    }  
}
```

Result:

Conclusion:

Questions:

Experiment No . 09

Aim: To interface an 8-bit ADC (e.g., ADC0808/0809) with LCD module using the 8051 microcontroller and display the converted digital value on LCD.

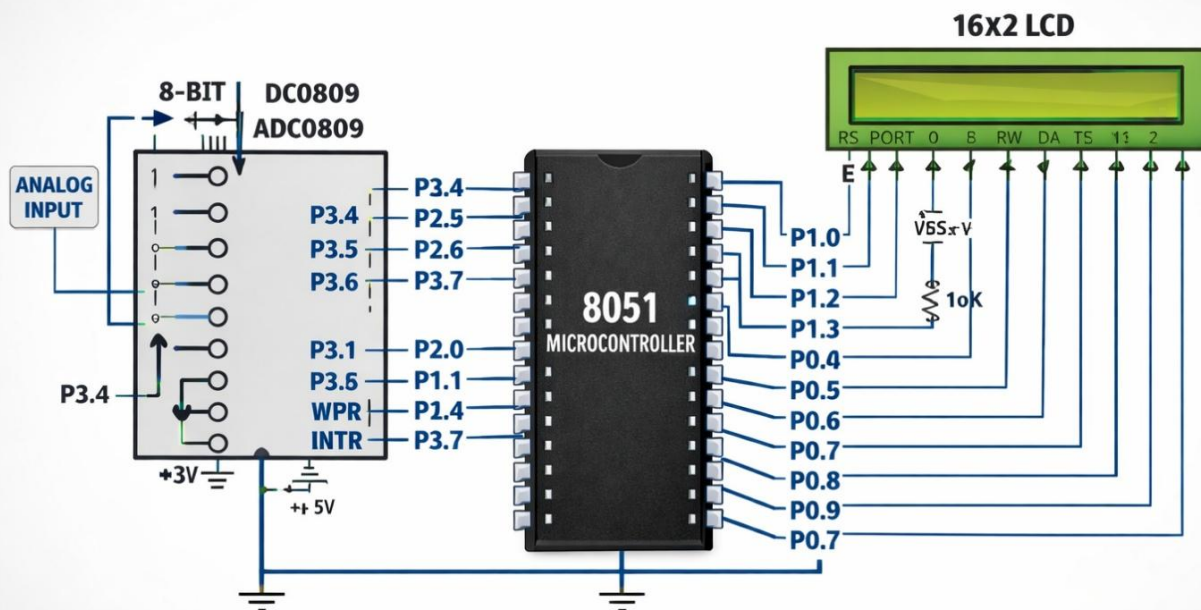
Apparatus Required: 8051 Microcontroller Kit / Development Board, FRC cables, 16×2 LCD Module, 8-bit ADC0808/0809 IC, 12V Power supply

Theory:

An ADC is an electronic system that changes analog signals (continuous values) into digital signals (discrete binary numbers) so that a microcontroller or computer can understand and process them.

- Analog signals come from real-world sources like temperature sensors, sound, or light, and they can have many values.
- Digital signals are stored and processed in binary form (0s and 1s).
- ADC works by sampling the analog input and quantizing it into fixed levels, then converting those levels into binary output.
- The number of bits (like 8-bit, 10-bit, 12-bit) decides how precise the digital output will be.
 - Example: 8-bit ADC → 256 levels (2^8)
- ADC conversion time is very small, so it supports real-time data reading.
- Common types include:
 1. Flash ADC – very fast, less used for high accuracy
 2. Successive Approximation (SAR) ADC – fast + accurate, popular
 3. Dual Slope ADC – high accuracy, slower speed
- Many ADC ICs give an End of Conversion signal (EOC/INTR) to tell the controller that the digital value is ready to be read.

INTERFACING 8-BIT ADC & LCD WITH 8051



INTERFACING 8-BIT ADC & LCD WITH 8051

Algorithm:

1. Start
2. Initialize LCD module
3. Configure PORT2 = 0xFF (input) for ADC data
4. Configure control pins on PORT3
5. Display static text on LCD ("8-bit ADC Test", etc.)
6. Send LOW pulse on CS and WR to start conversion
7. Wait until INTR becomes LOW (conversion complete)
8. Make CS LOW and RD LOW, read 8-bit value from PORT2
9. Convert 8-bit value into hex ASCII
10. Send characters to LCD
11. Repeat forever

PROCEDURE:

1. *Connect +12V supply to board.*
2. *Connect the USB Cable to the micro controller Kit and select COM port of the PC.*
3. *Switch ON the power supply.*
4. *Open ProgISP Flashing Software on PC. Select the IC (AT89S52).*
5. *Select given .hex File -> click on Load Flash. Select the HEX file. Click on Auto Button. This uploads HEX file into microcontroller shown by green bar.*
6. *Connect 10 pin FRC Cables to the 8 Bit ADC section of the DVK kit. Press RESET and observe output.*
7. *Observe the output of LEDs (00 to FF) through the POT (0 to 5 V).*

Note: Keep link at SL4 between 1 &2, SL5 between 2,3 short

Result:**Conclusion:****Questions:**

1. ADC stands for?
2. An 8-bit ADC can represent how many levels?
3. Which signal shows conversion is complete in many ADC ICs?
4. Fastest ADC type among the following — SAR, Flash, Dual Slope?

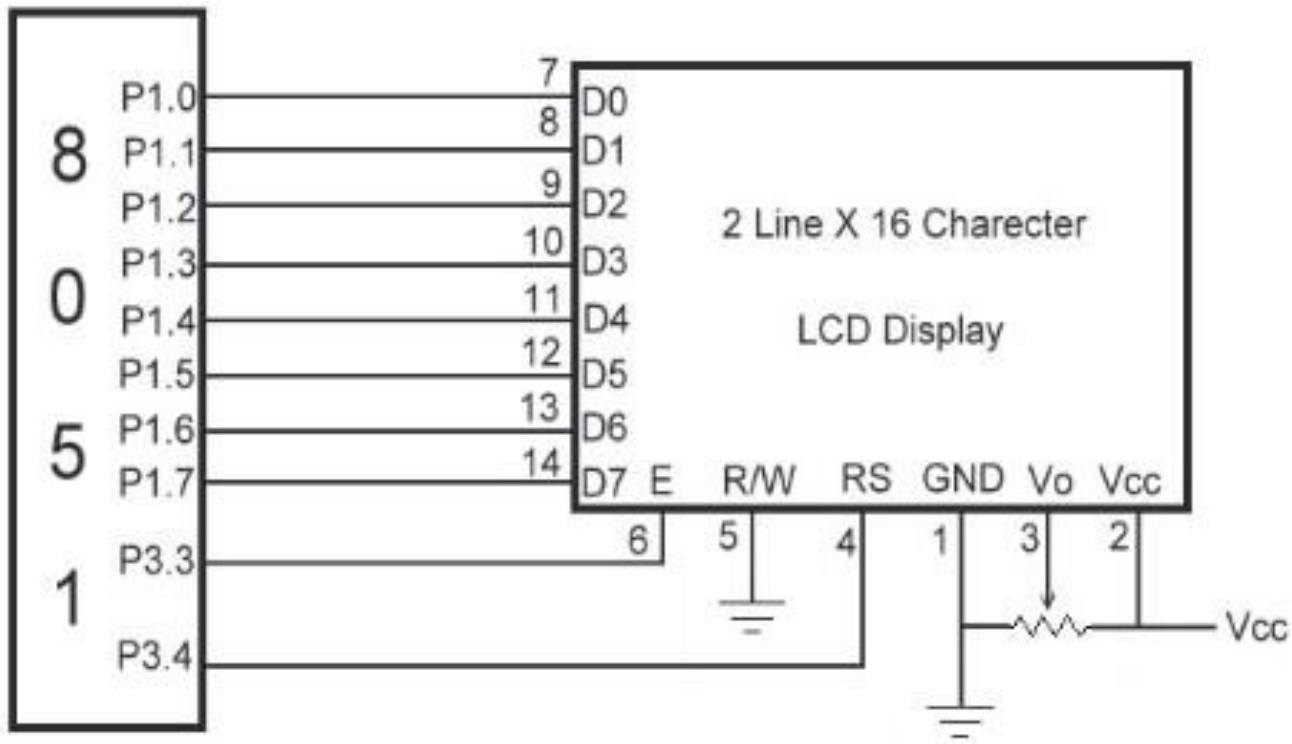
Experiment No . 10

Aim: To write a “C” language program to interface LCD Display with Microcontroller.

Apparatus Required: 8051 Microcontroller kit, FRC cables

Software Used: Keil Micro vision 4

Circuit Diagram:



Theory:

Interfacing an LCD (Liquid Crystal Display) with the 8051 microcontroller is a common task in embedded systems. The following example demonstrates how to connect a 16x2 LCD display to an 8051 microcontroller and write a simple program to display characters.

Hardware Connections

1. LCD Pins:

- VSS (Pin 1): Ground
- VDD (Pin 2): +5V power supply
- VEE (Pin 3): Contrast adjustment (connect to a potentiometer)
- RS (Pin 4): Register Select (0: Command register, 1: Data register)
- RW (Pin 5): Read/Write (0: Write, 1: Read)
- E (Pin 6): Enable

- D0-D7 (Pins 7-14): Data lines
- LED+ (Pin 15): Backlight Anode
- LED- (Pin 16): Backlight Cathode

2. Connecting to 8051:

- Connect VSS to Ground.
- Connect VDD to +5V.
- Connect VEE to the wiper of a potentiometer for contrast adjustment.
- Connect RS, RW, and E to specific port pins of the 8051.
- Connect D4-D7 to specific port pins of the 8051 (in 4-bit mode).
- Connect the backlight anode to +5V and the cathode to Ground through a current-limiting resistor.

Pin Assignment Example

- RS: P2.0
- RW: P2.1
- E: P2.2
- D4-D7: P2.4-P2.7

Algorithm

1. Start
2. Include 8051 register header and LCD library.
3. Define LCD control pins:
 - RS, RW, EN as specific Port 1 bits.
4. Configure Port 0 and Port 1 as output ports.
5. Initialize ports to 00H (clear previous data).
6. Call lcd_init() to initialize LCD.
7. Inside lcd_init():
 1. Send command 38H → 2 line, 8-bit mode.
 2. Give delay.
 3. Send command 0CH → Display ON, cursor OFF.
 4. Give delay.
 5. Send command 06H → Auto increment cursor.
 6. Give delay.
 7. Send command 01H → Clear screen.
 8. Give delay.
 9. Send command 80H → Move cursor to line 1 start.

8. Create function to send LCD command (lcd_cmd):
 - Load data on Port 0.
 - Set RW=0, RS=0.
 - Pulse enable pin using enable().
9. Create function to send LCD data (lcd_dat):
 - Load character on Port 0.
 - Set RW=0, RS=1.
 - Pulse enable pin.
10. Display strings using lcd_dis(row, string):
 - If row 1 → command 80H.
 - Else row 2 → command C0H.
 - Send characters one by one until string ends.
 - Count characters sent.
 - If characters <16 → pad remaining places with blank spaces.
11. Enable LCD by pulsing EN HIGH then LOW with small delay.
12. Provide delay using delaysms() function (nested empty loops).
13. Run infinite loop to hold display output.
14. End

Procedure:

1. Connect +12V supply to board.
2. Connect the USB to the micro controller Kit and select COM port of the PC.
3. Switch ON the power supply.
4. Open ProgISP Flashing Software on PC. Select the IC (AT89S52).
5. Select given .hex File -> click on Load Flash. Select the HEX file. Click on Auto Button. This uploads HEX file into microcontroller shown by green bar.
6. Now if downloading of the program is completed, connect 10 pin FRC cables to the
7. LCD section of the DVK kit
8. You will observe LOGSUN SYSTEMS.

Connections:**LCD(4-bit):**

PORT 0(J1) To J9(LCD)

LCD(8-BIT)

PORT 0(J1) To J9(LCD)

PORT 1(J2) To J10(LCD)

Result:**Conclusion:****Questions:**

1. Which ports of 8051 are used for sending LCD data and LCD control signals in the above circuit?
2. What is the purpose of the command 0x38 in the lcd_init() function?
3. Why is RW set to 0 during both command and data operations?
4. What is the function of the EN (Enable) pin in LCD interfacing?

